

These are important concepts in AI model optimization, especially when running LLMs locally with tools like Ollama or on GPUs using frameworks from NVIDIA.

1. What is Quantization?

Quantization means:

👉 **Reducing the precision of model numbers to make models smaller and faster**

LLMs store billions of weights (numbers).

Normally:

32-bit floating point (FP32)

Quantization converts them into:

16-bit, 8-bit, 4-bit, etc.

Why quantization is used

Large models are huge.

Example:

- FP32 model → 28 GB
- 4-bit quantized → ~4 GB

Benefits:

- Less RAM/VRAM
 - Faster inference
 - Cheaper deployment
-

Simple analogy

Imagine storing temperatures:

32.48719283

Instead of full precision, store:

32.5

Slightly less accurate, much smaller.

Before vs after quantization

Precision	Size	Speed	Accuracy
FP32	Very large	Slow	Best
FP16	Smaller	Faster	Very good
INT8	Much smaller	Fast	Slight drop
INT4	Tiny	Very fast	More quality loss

2. What is Calibration?

Calibration is:

👉 **The process of measuring how model values behave before quantizing**

The system checks:

- weight ranges
- activation ranges
- important value distributions

This helps decide:

- how aggressively to compress
 - how to minimize accuracy loss
-

Why calibration matters

Without calibration:

Important values may get distorted

Result:

- hallucinations
 - worse answers
 - unstable output
-

Simple analogy

Suppose you compress a photo.

Calibration decides:

“Which parts of the image need high quality?”

so important details are preserved.

3. Modes / Types of Quantization

There are several major modes.

◆ A. Dynamic Quantization

Quantization happens during inference.

Characteristics:

- Easy
- Fast setup
- Moderate optimization

Best for:

- CPUs
 - Simple deployments
-

◆ B. Static Quantization

Model is calibrated beforehand.

Steps:

1. Run calibration dataset
2. Measure activations
3. Quantize permanently

Better performance than dynamic.

◆ C. Quantization-Aware Training (QAT)

The model is trained while simulating quantization.

Best accuracy after compression.

But:

- more expensive
- requires retraining



Comparison

Type	Accuracy	Complexity	Speed
Dynamic	Medium	Easy	Good
Static	Better	Medium	Better
QAT	Best	Hard	Best



4. Common LLM Quantization Formats

When using local LLMs (like in Ollama), you'll often see:

Format	Meaning
Q4	4-bit quantization
Q5	5-bit
Q8	8-bit
GPTQ	GPU optimized quantization
AWQ	Activation-aware quantization
GGUF	Popular local LLM format



Example

llama3:8b-q4_K_M

Means:

- Llama 3
- 8B parameters

- Quantized to 4-bit
-

Trade-off (very important)

Quantization always balances:

Smaller + Faster ↔ Accuracy

Too much quantization:

- weaker reasoning
 - hallucinations
 - poorer coding ability
-

Real-world analogy

- FP32 = RAW DSLR photo 
- INT4 = compressed JPEG 

Smaller and faster, but some detail lost.

Bottom line

Quantization

👉 Compress model numbers for faster/smaller inference

Calibration

👉 Measure model behavior before compression

Quantization modes

👉 Different strategies for balancing speed vs accuracy

If you want, I can also explain:

- 🔥 GGUF vs GPTQ vs AWQ
- ⚡ why Ollama models use quantization heavily
- 🧠 how quantization affects RAG/chatbot quality
- 📊 visual memory savings by quantization level

The key difference is **when quantization happens**.

One-line summary

- **Post-Training Quantization (PTQ)**
→ quantize **after** the model is already trained
 - **Quantization-Aware Training (QAT)**
→ train the model while simulating quantization
-

Side-by-side comparison

Feature	PTQ	QAT
Quantization timing	After training	During training
Retraining needed?	❌ No	✅ Yes
Complexity	Easy	Hard
Accuracy	Lower	Higher
Cost	Cheap	Expensive
Best for	Fast deployment	Production optimization



1. Post-Training Quantization (PTQ)

Workflow:

Train model normally



Quantize model afterward

Example:

FP32 → INT8

No retraining needed.



Why PTQ is popular

Because it's:

- fast
- simple
- cheap

Very common in:

- local LLMs
 - Ollama models
 - edge devices
-



Downside

The model never “learned” how to behave under low precision.

So:

- accuracy may drop
- reasoning quality may degrade

especially at:

- 4-bit
 - 2-bit quantization
-

2. Quantization-Aware Training (QAT)

Workflow:

During training:
simulate quantization effects



Model adapts to low precision

The model sees fake quantization noise while training.

Why QAT performs better

The model learns:

“I will eventually run in low precision”

So it adapts its weights accordingly.

Result:

- much better accuracy
 - less degradation
-

Simple analogy

PTQ

Like:

Compressing a finished JPEG afterward

QAT

Like:

Taking the photo while already optimizing for compression



Visual intuition

PTQ

Normal training



Sudden compression



Possible quality loss

QAT

Training WITH compression simulation



Model adapts naturally



Higher quality after compression



Trade-off

PTQ

- ✓ Easy
 - ✓ Fast
 - ✗ More accuracy loss
-

QAT

- ✓ Better performance
 - ✓ Better low-bit quality
 - ✗ Expensive retraining
 - ✗ More complex pipeline
-

In LLM world

Most downloadable local models:

- use PTQ variants
- because retraining giant LLMs is expensive

Examples:

- GPTQ
 - AWQ
 - GGUF quantizations
-

Where QAT is common

QAT is more common in:

- mobile AI
- production inference systems
- enterprise deployments
- custom-trained models

especially when:

- latency matters

- model quality must stay high
-



Example

Suppose:

- original model = FP32

After quantization:

Method	Accuracy
PTQ INT4	82%
QAT INT4	89%

QAT preserves more performance.



Bottom line

Post-Training Quantization (PTQ)

- 👉 Compress after training
- 👉 Easy and common

Quantization-Aware Training (QAT)

- 👉 Train with quantization simulation
 - 👉 Harder but higher quality
-

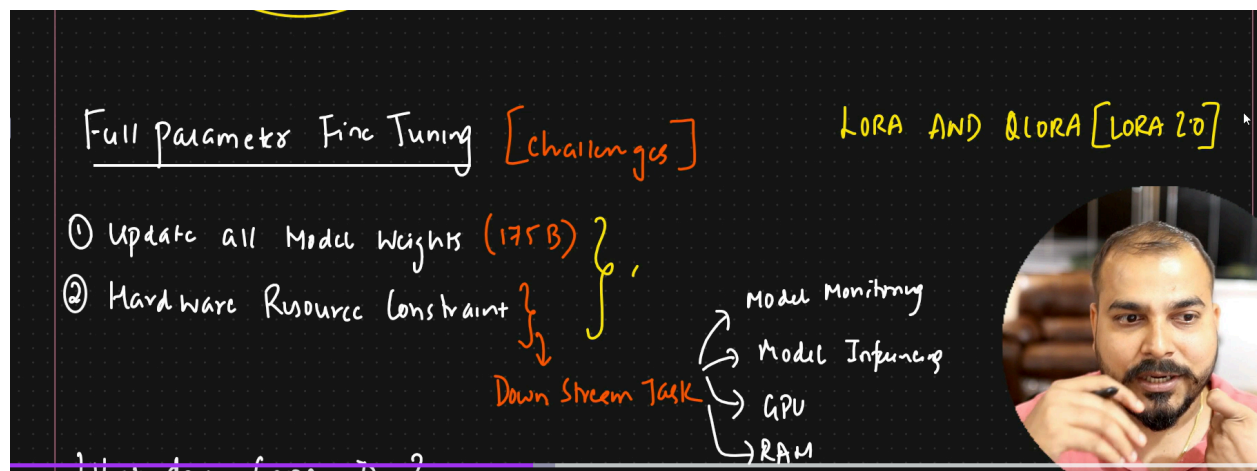
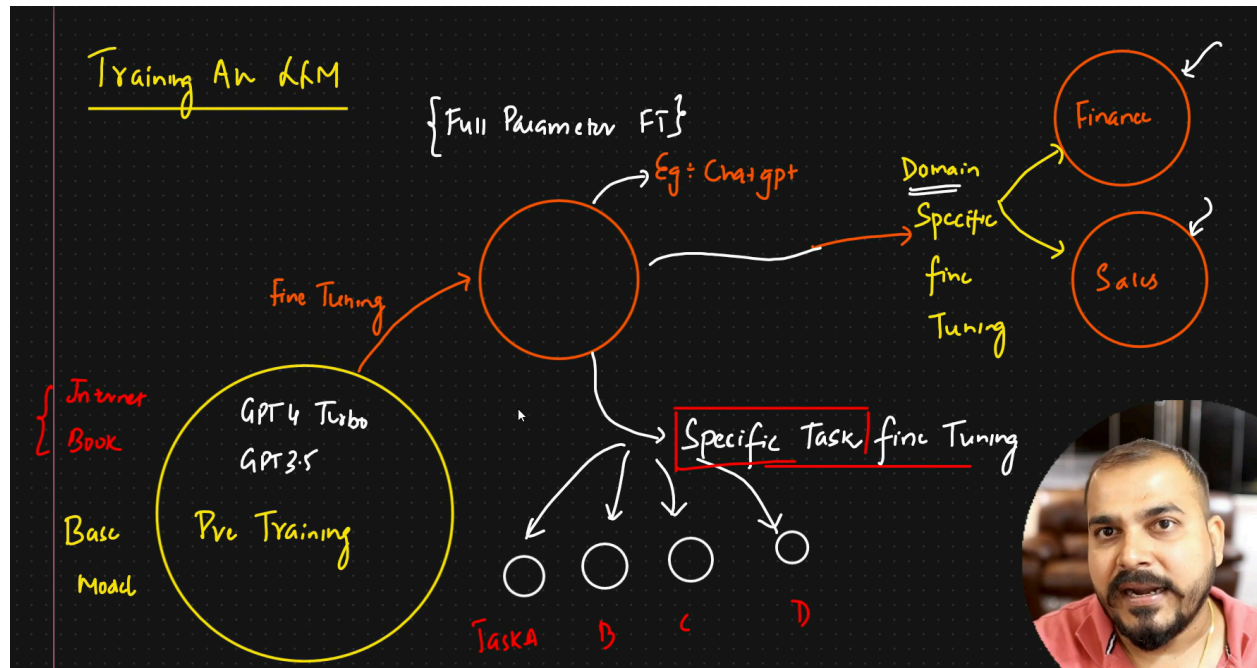


Mental shortcut

PTQ = compress later

QAT = train for compression

LORA and QLORA Indepth Mathematical intuition



What does LoRA Do?

→ RAM

① Instead of updating weights, it track changes.

Pre Trained Weights

3x3

+

LoRA Tracked weights

1	2	3
4	5	6
7	8	9

3x3

=

Fine Tuned Weights ←

W_0

+

ΔW

=

$W_0 + BA$

--	--	--

3x3

7	8	9
---	---	---

3x3

--	--	--

W_0

+

ΔW

=

$W_0 + BA$

Matrix Decomposition

B

1
2
3

3x1

x

A

4	5	6
---	---	---

1x3

{ How to Select D.
it Matters

Rank=1

Number of Trainable Parameters

Method	Hyperparameters	# Trainable Parameters
Fine-Tune	-	175B ✓
<u>PrefixEmbed</u>	$l_p = 32, l_i = 8$	0.4 M
	$l_p = 64, l_i = 8$	0.9 M
	$l_p = 128, l_i = 8$	1.7 M
	$l_p = 256, l_i = 8$	3.2 M
	$l_p = 512, l_i = 8$	6.4 M
<u>PrefixLayer</u>	$l_p = 2, l_i = 2$	5.1 M
	$l_p = 8, l_i = 0$	10.1 M
	$l_p = 8, l_i = 8$	20.2 M
	$l_p = 32, l_i = 4$	44.1 M
	$l_p = 64, l_i = 0$	76.1 M
<u>Adapter^H</u>	$r = 1$	7.1 M
	$r = 4$	21.2 M
	$r = 8$	40.1 M
	$r = 16$	77.9 M
	$r = 64$	304.4 M
LoRA	$r_v = 2$	4.7 M
	$r_q = r_v = 1$	4.7 M
	$r_q = r_v = 2$	9.4 M
	$r_q = r_k = r_v = r_o = 1$	9.4 M
	$r_q = r_v = 4$	18.8 M
	$r_q = r_k = r_v = r_o = 2$	18.8 M
	$r_q = r_v = 8$	37.7 M
	$r_q = r_k = r_v = r_o = 4$	37.7 M
	$r_q = r_v = 64$	301.9 M
	$r_q = r_k = r_v = r_o = 64$	603.8 M

→ Matrix Decomposition

No. of Trainable Parameter

Rank	7B	13B	70B	180B
→ 1	167K	228K	529K	849K
→ 2	334K	456K	1M	2M
<u>8</u>	1M	2M	4M	7M
<u>16</u>	3M	4M	8M	14M
<u>512</u>	86M	117M	270M	434M

High Rank



Model Complex

Things



What are LoRA and QLoRA?

Both are techniques for **fine-tuning large language models (LLMs)** cheaply and efficiently.

Instead of retraining the entire model (which is extremely expensive), they update only a **small number of parameters**.



The core problem they solve

A large model might have:

7 billion to 70 billion parameters

Training all of them:

- needs huge GPUs
- costs a lot
- takes days/weeks

So researchers created:

- **LoRA**
- **QLoRA**

to fine-tune models using far less memory.



1. What is LoRA?

Full form

LoRA = Low-Rank Adaptation

Main idea

Instead of changing the original model weights:

Original weights = frozen

LoRA adds:

- tiny trainable adapter layers

Only those adapters are trained.

Visual intuition

Normal fine-tuning:

Update ALL weights

LoRA:

Keep original model fixed

+

Train tiny adapters

Why LoRA is powerful

Benefits:

- Much less GPU memory
- Faster training
- Smaller checkpoint files
- Easy to share

Example:

- Full fine-tuning → hundreds of GB
 - LoRA → a few MB/GB
-



Simple analogy

Imagine modifying a huge textbook.

Instead of rewriting the whole book:

- you add sticky notes with corrections



Sticky notes = LoRA adapters



How LoRA works mathematically

LoRA approximates weight updates using:

$$\Delta W \approx AB$$

Where:

- (W) = original weights
- (A, B) = small low-rank matrices

Instead of learning a huge matrix:

- learn two tiny matrices

That's why memory usage drops massively.



LoRA characteristics

Feature	LoRA
Trains full model?	✗ No
Updates base weights?	✗ Frozen
Memory usage	Low
Speed	Fast

Quality

Very good

2. What is QLoRA?

Full form

QLoRA = Quantized LoRA

It combines:

- **Quantization**
 - **LoRA**
-

Main idea

QLoRA:

1. Quantizes the base model (usually 4-bit)
 2. Keeps it frozen
 3. Trains LoRA adapters on top
-

Workflow

Large model



4-bit quantization



Add LoRA adapters



Train adapters only



Why QLoRA was a huge breakthrough

Before QLoRA:

- Fine-tuning large models required massive GPUs

After QLoRA:

- People could fine-tune:
 - 7B
 - 13B
 - even 65B models

on much smaller hardware.



LoRA vs QLoRA

Feature	LoRA	QLoRA
Base model precision	FP16/FP32	Quantized (4-bit)
Memory usage	Low	VERY low
Speed	Fast	Fast
GPU requirements	Moderate	Minimal
Training quality	Excellent	Excellent



Simple analogy

LoRA

- Use sticky notes on a normal textbook

QLoRA

- Compress the textbook first
 - Then add sticky notes
-

Why QLoRA matters for local AI

Frameworks like:

- Ollama
- Hugging Face ecosystem

rely heavily on quantization.

QLoRA enables:

- affordable fine-tuning
 - consumer GPU training
 - local customization
-

Real-world example

Suppose you want:

“A medical chatbot based on Llama”

Without LoRA:

- retrain billions of parameters

With LoRA/QLoRA:

- train only tiny adapters on medical data

Much cheaper.

Important distinction

LoRA/QLoRA are:

- NOT inference methods
- NOT vector databases
- NOT RAG systems

They are:

👉 **fine-tuning techniques**

Bottom line

LoRA

👉 Train small adapters instead of whole model

QLoRA

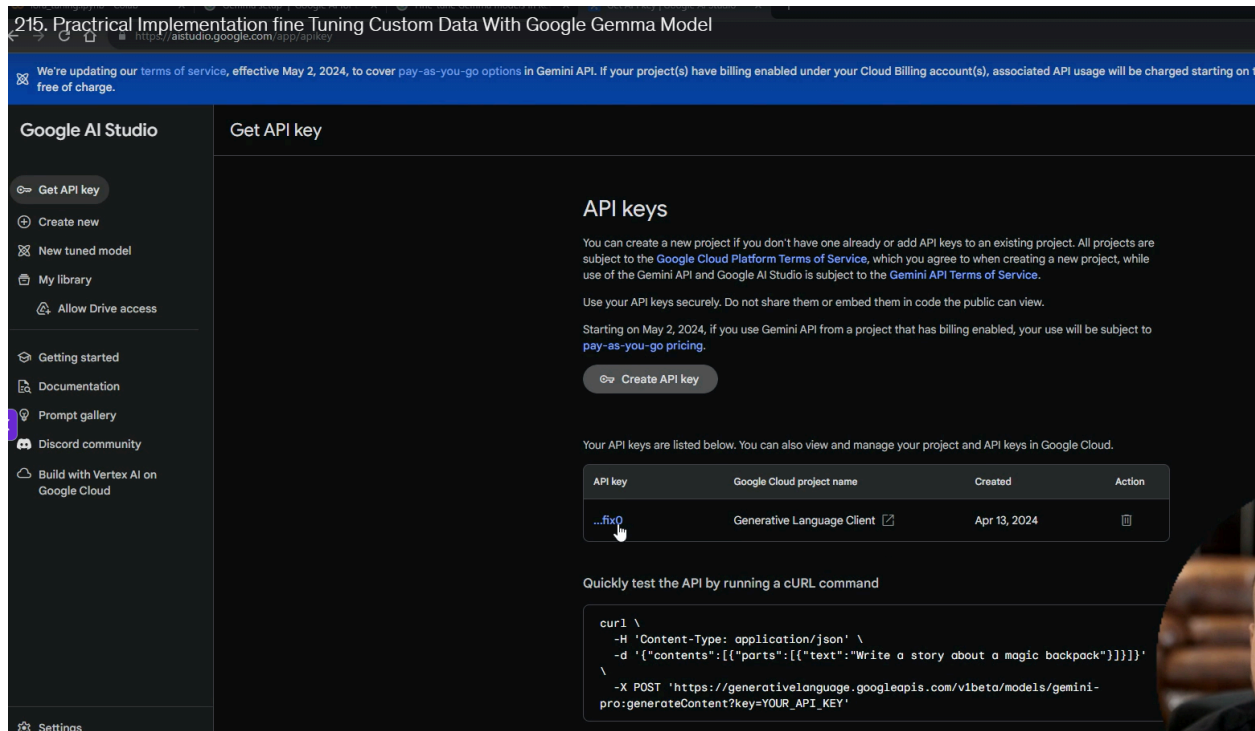
👉 Quantize the model first, then use LoRA

Mental shortcut

LoRA = lightweight fine-tuning

QLoRA = compressed lightweight fine-tuning

Practical implementation fine tuning custom data with google Gemma model



Both **Keras** and **Kaggle** are popular in AI/ML, but they serve completely different purposes.

One-line difference

- **Keras** → tool/library for building neural networks
- **Kaggle** → platform for learning, datasets, and ML competitions

Side-by-side comparison

Feature	Keras	Kaggle
Type	Deep learning library	ML platform/community
Purpose	Build/train models	Learn/share/compete
Used for	Coding AI models	Practicing ML
Runs models?	✓ Yes	✗ Not itself

Provides
datasets?



✓ Huge collection

Competitions?



✓ Yes



1. What is Keras?

Keras is a **high-level deep learning API**.

It helps you build:

- neural networks
- image models
- NLP models
- AI systems

very easily.

Today, Keras is tightly integrated with:

- TensorFlow
-



Example Keras code

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(64, activation='relu'),
    layers.Dense(10)
])
```

This creates a neural network.



Why Keras became popular

Before Keras:

- deep learning code was complicated

Keras made it:

- cleaner
 - simpler
 - beginner-friendly
-

Simple analogy

Keras is like:

LEGO blocks for neural networks 

2. What is Kaggle?

Kaggle is an online platform for:

- datasets
- competitions
- notebooks
- learning ML

Owned by:

- [Google](https://www.google.com/)
-

What people do on Kaggle

Use datasets

Example:

- housing prices
 - medical images
 - NLP datasets
-

Join competitions

Example:

“Predict customer churn”

People compete for:

- rankings
 - money
 - reputation
-

Run notebooks in cloud

You can:

- write Python
- train models
- use free GPUs

without setting up your own machine.

Simple analogy

Kaggle is like:

GitHub + LeetCode + Coursera for ML

How they work together

This is common:

Kaggle → get dataset



Keras → build model



Train + submit predictions



Example workflow

On Kaggle:

1. Download image dataset
 2. Use Keras CNN model
 3. Train model
 4. Submit predictions
-



Important distinction

Keras

= coding framework

Kaggle

= learning/community platform

They are not competitors.



When to use what

Use Keras if:

- You want to build AI models
 - Learn deep learning
 - Train neural networks
-

Use Kaggle if:

- You want datasets
 - Practice ML
 - Learn from notebooks
 - Join competitions
-



Bottom line

- **Keras = build neural networks**
 - **Kaggle = practice and learn ML with datasets/competitions**
-

If you want, I can also explain:

- TensorFlow vs Keras
- PyTorch vs Keras
- how Kaggle notebooks work
- beginner roadmap for ML using Kaggle + Keras

Complete Generative AI Course With Langchain and Huggingface

Gemma
keras/gemma

Keras implementation of the Gemma model. This Keras 3 implementation will run on JAX, TensorFlow and PyTorch.

Model Card Code (251) Discussion (5)

You've consented to the license for Gemma

Model Details

Gemma Model Card

Model Page: Gemma

Resources and Technical Documentation:

- Responsible Generative AI Toolkit
- Gemma on Kaggle
- Gemma on Vertex Model Garden

colab.research.google.com

lora_tuning.ipynb

File Edit View Insert Runtime Tools Help Cannot save changes

Secrets

Notebook access	Name	Value	Actions
☒	GOOGLE_API_KEY		
☐	HF_TOKEN		
☒	KAGGLE_KEY		
☒	KAGGLE_USER		
☐	groq_api_key		

+ Add new secret

credentials.

In Colab, select **Secrets** (🔑) in the left pane and add a new key. Store your username under the name **KAGGLE_KEY**.

Set environment variables

Set environment variables for KAGGLE_KEY

```
[ ] import os
    from google.colab import secrets

    # Note: `userdata.get` is used to retrieve secrets.
    # vars as appropriate for the environment.
```

[Databricks](https://databricks.com) is a cloud platform used for:

- big data processing
- data engineering
- machine learning
- AI analytics

all in one place.

Simple definition

👉 Databricks helps companies:

- store massive data
- process it
- train AI models
- analyze business data

using distributed computing.

Why Databricks became popular

Before Databricks, companies often had:

- separate systems for:
 - data engineering
 - analytics
 - ML

Databricks unified them.

Core technologies behind Databricks

Databricks was created by the team behind:

- Apache Spark

Spark is the engine that processes huge datasets across clusters of machines.

What Databricks is mainly used for

1. Big data processing

Example:

- billions of rows of logs
 - financial transactions
 - clickstream data
-

2. Data engineering

Build ETL pipelines:

Raw Data
↓
Clean/Transform
↓
Analytics / ML

3. Machine learning

Train models at scale.

Supports:

- TensorFlow
 - PyTorch
 - scikit-learn
-

4. AI / LLM workflows

Modern Databricks also supports:

- vector search
- RAG
- model serving
- AI agents



Databricks vs traditional database

Feature	Databas e	Databricks
Stores data	✓	✓
Processes huge distributed data	Limited	✓
ML workflows	Limited	✓
Spark integration	✗	✓
AI pipelines	Basic	Advanced



Simple analogy

- SQL database = spreadsheet 
 - Databricks = entire data factory 
-



Important Databricks concepts

Lakehouse Architecture

Databricks popularized:

- **Data Lake + Data Warehouse**

combined into:

Lakehouse

Delta Lake

A storage layer for:

- reliability
 - versioning
 - fast queries
-



Example real-world usage

Suppose Netflix-like company has:

- user clicks
- watch history
- recommendations

Databricks can:

1. process petabytes of data
 2. train recommendation models
 3. serve analytics dashboards
-



Important distinction

Databricks is NOT:

- just a database
- just an AI tool
- just cloud storage

It's a:

👉 **data + AI platform**



Relationship with AI ecosystem

Databricks often works with:

- TensorFlow
- PyTorch
- Hugging Face models
- LangChain
- vector databases



When to use Databricks

Use it if you need:

- massive-scale data processing
- enterprise analytics
- distributed ML pipelines
- AI infrastructure



Overkill for:

- small personal projects
- simple chatbots
- lightweight apps



Bottom line

👉 Databricks = enterprise platform for big data + machine learning + AI at scale.

The last part of the video is not clear